

DISTRIBUIRANI SISTEMI

TEORIJA:

- Šta predstavlja skalabilnost distribuiranog sistema? Navesti i objasniti tehnike skaliranja distribuiranih sistema.
- Šta se podrazumeva pod transparentnošću konkurencije:
 - sve niti mogu pristupati deljivim strukturama podataka
 - procesu mogu pristupati resursima bez međusobne interferencije
 - repliciranim resursima se pristupa kao da postoji samo jedna kopija
 - novi čvorovi se mogu dodati sistemu bez promene aplikacije
- Izlaz iz Sun IDL kompajlera sastoji se od više fajlova. Koji su to fajlovi i šta sadrže? Napisati na koji način se vrši generisanje ovih fajlova i kako se na osnovu generisanih fajlova formiraju izvršna klijentska i serverska aplikacija. Za svaki korak napisati odgovarajuću komandu.
 - Koja semantika poziva udaljenih procedura je podržana kod DCE RPC i kako?
- Navesti kriterijume za podelu komunikacija u distribuiranim sistemima. Koji sve tipovi komunikacija u distribuiranom sistemu su podržani od strane MPI i kojim funkcijama? Obrazložiti izvršenje svake funkcije.
 - Šta predstavlja JMS administrativni objekat, koja je njegova uloga i koji su primeri takvih objekata? Iz čega se sastoji JMS poruka i kako i na osnovu čega se može vršiti filtriranje poruke u JMS?
- Šta je definicija potpuno uređene grupne komunikacije:
 - poruke se isporučuju procesima u FIFO redosledu
 - poruke se procesima isporučuju po redosledu realnog vremena
 - poruke se isporučuju procesima po redosledu „desilo se pre“
 - poruke se isporučuju svim procesima po istom redosledu
- Tri procesa P1, P2 i P3 izvršavaju instrukcije nad tri deljive promenljive x, y i z. Postoje dve replike R1 i R2 u kojima se pamte promenljive x, y i z. Promenljive x, y i z su inicijalizovane na nulu.

P1	P2	P3
x=1 print (y,z)	y=1 print (x,z)	z=1 print (x,y)

Operacije na replikama R1 i R2 se izvode u sledećem redosledu:

R1	R2
x=1 print (y,z) y=1 print (x,z) z=1 print (x,y)	x = 1 y = 1 print (x,z) print (y,z) z = 1 print (x,y)

Da li je ovakvo skladište podataka sekvencijalno konzistentno? Obrazložiti odgovor

- Koji tipovi grešaka u odnosu na trajanje postoje? Dati primer za svaku od njih. Objasniti razliku između “mirnih” i Vizantijskih grešaka.
 - U grupi postoji deset repliciranih procesa. Ako mogu nastupiti samo mirne greške, koliko maksimalno procesa može otkazati a da se ipak dobije korektan rezultat? Ako mogu nastupiti greške vizantijskog tipa, koliko procesa može maksimalno otkazati a da se ipak dobije korektan rezultat. Šta ako mogu nastupiti greške vizantijskog tipa a procesi moraju postići konsenzus? Koliko u ovom slučaju maksimalno procesa može otkazati? Svaki odgovor obrazložiti.

8.

- a) Kada smo govorili o DFS rekli smo da server može biti projektovan kao statefull ili stateless. Pored svakog od tvrđenja staviti oznaku tačno (T) ili netačno (F).
 - 1) Implementacija klijentske strane može biti komplikovanija sa statefull serverom
 - 2) Zaključavanje fajla je teško implementirati kod stateless servera
 - 3) Kod statefull servera, svaki klijentski zahtev mora da sadrži kompletnu informaciju o zahtevu (npr. ime fajla, offset, itd)
 - 4) Lakše je izabrati se sa greškama kod stateless nego kod statefull servera
- b) Navesti sve demone u Hadoop klasteru, objasniti njihove uloge kao i gde se izvršavaju u Hadoop klasteru. Šta predstavlja blok i koje su prednosti korišćenja blokova kod HDFS? Pretpostavimo da je fajl veličine 514MB sačuvan na HDFS-u. Ako je veličina bloka 64MB i podrazumevani faktor replikacije 4, koliki je ukupni broj blokova i kolika je veličina svakog od njih?

ZADACI:

1. U .NET-u, koristeći gRPC, kreirati servis za upravljanje listom poruka. Servis treba da omogući klijentima da dodaju i brišu poruke, i prikažu sve poruke.

Definisati proto fajl (message.proto) koji daje specifikaciju servisa i poruka. Servis treba da podržava sledeće operacije:

- SendMessage: dodaje novu poruku na listu.
- DeleteMessage: briše poruku sa zadatim ID-jem.
- ListMessages: Dobije tok podataka koji sadrži sve poruke.

Implementirati gRPC server definisan u message.proto fajlu.

2. Korišćenjem Java RMI tehnologije napisati simulaciju MQTT brokera. MQTT broker je server koji razmenjuje poruke između klijenata korišćenjem topika. Topik predstavlja *string* na osnovu koga se poruke (definisane naslovom i sadržajem) filtriraju i dostavljaju određenim klijentima. Neophodno je implementirati metodu *subscribe* kako bi se klijent pretplatio na poruke pristigle na određeni topik, kao i metodu *publish* koja omogućava korisniku da pošalje poruku na željeni topik. Klijent ne kreira topik eksplicitno, već se topik, ukoliko ne postoji, kreira implicitno pozivom metode *createTopic*. Napisati serversku aplikaciju za hostovanje MQTT brokera i registrovati ga u RMI registar, kao i klijentsku aplikaciju koja će minimalno demonstrirati funkcionisanje istog.

3. Napisati MPI program koji vrši paralelni upis i čitanje binarne datoteke, prema sledećim zahtevima:

- a) Svaki proces upisuje po 105 proizvoljnih celih brojeva u datoteku file1.dat. Upis se vrši upotrebom pojedinačnih pokazivača, dok redosled podataka u fajlu ide od podataka poslednjeg do podataka prvog procesa.
- b) Ponovo otvoriti datoteku. Svaki proces vrši čitanje upravo upisanih podataka upotrebom funkcija sa eksplicitnim pomerajem.
- c) Upravo pročitane podatke upisati u novu datoteku, na način prikazan na slici (za slučaj od 3 procesa).



U poslednjem zahtevu posebno obratiti pažnju na efikasnost paralelizacije upisa.

4. Koristeći WCF kreirati sistem za zakup skladišta. Potrebno je da servis podržava sledeće funkcionalnosti:

- Zakup skladišta (proslediti Vlasnika(ime, prezime i jmbg) i skladište(IdSkladišta, početak zakupa, kraj zakupa, cena).
- Vraća listu svih aktivnih skladišta zadanog vlasnika.
- Vraća listu svih vlasnika aktivnih skladišta.
- Vraća listu svih skladišta i istoriju njihovih vlasnika (zakupa).

Obavezno izdvojiti interfejs, implementaciju, web.config (dovoljan je samo deo za setovanje servisa) i klijentsku stranu koja demonstrira rad servisa.